Date: 12/03/26
Docker Compose
Example 4. Deploy Node.js + MongoDB Application

Objective

1. To understand multi-container applications using Docker Compose.
2. To deploy a Node.js web application connected with a MongoDB database.

Project Structure

Create the following folder structure:

```
node-mongo-compose/
│
├── docker-compose.yml
├── package.json
├── server.js
└── Dockerfile
```

file name : server.js

```
const express = require("express");
const mongoose = require("mongoose");
const app = express();
const PORT = 3000;
mongoose.connect("mongodb://mongo:27017/mydatabase", {
useNewUrlParser: true,
useUnifiedTopology: true
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
app.get("/", (req, res) => {
res.send("Node.js + MongoDB running with Docker Compose");
});
app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
```

```js
JS server.js > ...
const express = require("express");
const mongoose = require("mongoose");
const app = express();
const PORT = 3000;
mongoose.connect("mongodb://mongo:27017/mydatabase", {
useNewUrlParser: true,
useUnifiedTopology: true
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
app.get("/", (req, res) => {
res.send("Node.js + MongoDB running with Docker Compose");
});
app.listen(PORT, () => {
console.log(`Server running on port ${PORT}`);
});
```

This is a simple backend logic written in express js

File name : package.json
These are the dependencies

```
{
"name": "node-mongo-app",
"version": "1.0.0",
"main": "server.js",
"dependencies": {
"express": "^4.18.2",
"mongoose": "^7.0.0"
}
}
```

Dockerfile (for Node.js App)

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

```
dockerfile > ...
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

FROM → describes which base image to use, also pulls images from the local, dockerhub registry.
WORKDIR → It sets the working directory for all the subsequent instructions such as ADD, COPY, ENTRYPOINT.
COPY → it copies from host system to the container
RUN → it executes command while building the image
EXPOSE → EXPOSE specifies the port a container listens on, but it does not publish the port. Port mapping is done using the -p flag.
CMD → defines what command to run when container starts

Docker Compose File
docker-compose.yml

```
version: '3.8'
services:
app:
build: .
container_name: node_app
ports:
- "3000:3000"
depends_on:
- mongo

restart: always
```

```yaml
mongo:
image: mongo:6
container_name: mongo_db
ports:
- "27017:27017"
volumes:
- mongo_data:/data/db
restart: always
volumes:
mongo_data:
```

docker-compose.yml

```yaml
Run All Services
services:
  Run Service
  node-app:
    build: .
    container_name: node_app
    ports:
      - "3000:3000"
    depends_on:
      - mongo
    restart: always

  Run Service
  mongo:
    image: mongo:6
    container_name: mongo_app
    ports:
    - "27017:27017"
    volumes:
      - mongo_data:/data/db
    restart: always
volumes:
  mongo_data:
```

Steps to Run the Application

Step : Start containers

docker compose up --build

```
sumitkumarmehta@sumits-MacBook-Air node-mango-compose % docker compose up --build
[+] Building 2.4s (13/13) FINISHED
 => [internal] load local bake definitions                                                    0.0s
 => => reading from stdin 602B                                                                0.0s
 => [internal] load build definition from dockerfile                                          0.0s
 => => transferring dockerfile: 146B                                                          0.0s
 => [internal] load metadata for docker.io/library/node:18                                    2.2s
 => [auth] library/node:pull token for registry-1.docker.io                                   0.0s
 => [internal] load .dockerignore                                                             0.0s
 => => transferring context: 2B                                                               0.0s
 => [1/5] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783   0.0s
 => => resolve docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783   0.0s
 => [internal] load build context                                                             0.0s
 => => transferring context: 130B                                                             0.0s
 => CACHED [2/5] WORKDIR /app                                                                 0.0s
 => CACHED [3/5] COPY package.json .                                                          0.0s
 => CACHED [4/5] RUN npm install                                                              0.0s
 => CACHED [5/5] COPY . .                                                                     0.0s
```

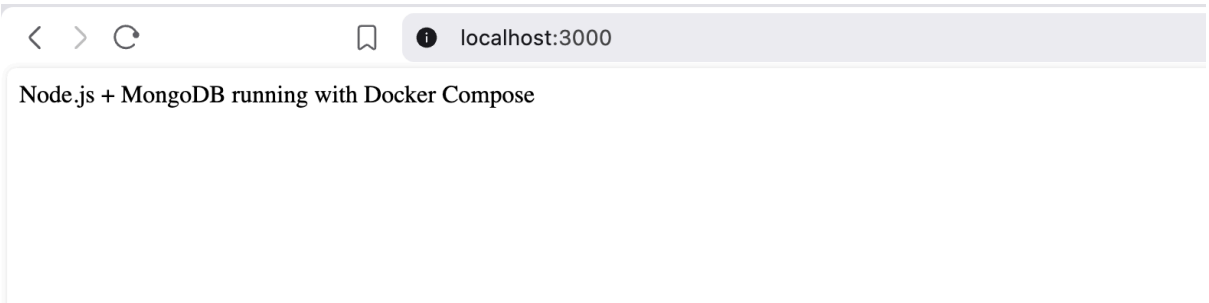
```
[+] up 4/4
 ✔ Image node-mango-compose-node-app  Built                                                   2.4s
 ✔ Network node-mango-compose_default Created                                                 0.0s
 ✔ Container mongo_app                Created                                                 0.0s
 ✔ Container node_app                 Created                                                 0.0s
Attaching to mongo_app, node_app
mongo_app  | {"t":{"$date":"2026-04-01T18:27:09.443+00:00"},"s":"I",  "c":"CONTROL",  "id":23285,   "ctx":"-","msg":"Automaticall
y disabling TLS 1.0, to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'"}
mongo_app  | {"t":{"$date":"2026-04-01T18:27:09.446+00:00"},"s":"I",  "c":"NETWORK",  "id":4915701, "ctx":"main","msg":"Initializ
ed wire specification","attr":{"spec":{"incomingExternalClient":{"minWireVersion":0,"maxWireVersion":17},"incomingInternalClient"
:{"minWireVersion":0,"maxWireVersion":17},"outgoing":{"minWireVersion":6,"maxWireVersion":17},"isInternalClient":true}}}
mongo_app  | {"t":{"$date":"2026-04-01T18:27:09.446+00:00"},"s":"I",  "c":"NETWORK",  "id":4648601, "ctx":"main","msg":"Implicit
TCP FastOpen unavailable. If TCP FastOpen is required, set tcpFastOpenServer, tcpFastOpenClient, and tcpFastOpenQueueSize."}
mongo_app  | {"t":{"$date":"2026-04-01T18:27:09.448+00:00"},"s":"I",  "c":"REPL",     "id":5123008, "ctx":"main","msg":"Successfu
lly registered PrimaryOnlyService","attr":{"service":"TenantMigrationDonorService","namespace":"config.tenantMigrationDonors"}}
mongo_app  | {"t":{"$date":"2026-04-01T18:27:09.448+00:00"},"s":"I",  "c":"REPL",     "id":5123008, "ctx":"main","msg":"Successfu
```

Step : Verify running containers

docker ps

```
sumitkumarmehta@sumits-MacBook-Air node-mango-compose % docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED              STATUS          PORTS
                                   NAMES
41d56069e195   node-mango-compose-node-app   "docker-entrypoint.s…"   About a minute ago   Up 3 seconds    0.0.0.0:3000->3000/tcp,
 [::]:3000->3000/tcp        node_app
1ecdeb4d4bd0   mongo:6                        "docker-entrypoint.s…"   About a minute ago   Up 3 seconds    0.0.0.0:27017->27017/tc
p, [::]:27017->27017/tcp   mongo_app
```

Test the Application

Open browser:

http://localhost:3000